Week 3 Reading: State Transitions and Trajectories

Trajectory prediction from initial states and future inputs

Approximate F_Δt, which maps current concentration and a held interval input to next concentration. Given c_0 and $T_0,\ldots,T_{149}$, recursively compute 150 future states.

$$c_{k+1} = F_{\Delta t}\left(c_k, u_k; q\right)$$

Fix Δt = 0.2 min, τ and C_Af within each trajectory. T changes between intervals. The ideal mixed state contains all three species; thermal dynamics are omitted.

Derive the concentration balances

For constant volume V and flow Q, τ = V/Q. Divide each species inventory balance by V. A enters at concentration C_Af and reacts into B; B reacts into C.

$$\frac{dC_A}{dt} = \frac{C_{\mathrm{Af}} - C_A}{\tau} - k_1 C_A$$
$$\frac{dC_B}{dt} = k_1 C_A - \frac{C_B}{\tau} - k_2 C_B$$
$$\frac{dC_C}{dt} = k_2 C_B - \frac{C_C}{\tau}$$

Adding the three equations cancels both reaction terms, giving dS/dt = (C_Af − S)/τ. Therefore S(t) = C_Af + [S(0) − C_Af] exp(−t/τ) under fixed feed.

S(0) = C_Af gives a constant sum. An arbitrary initial sum instead relaxes toward feed.

Derive the interval solution

$$\frac{dc}{dt} = Mc + b$$

$$M = \begin{bmatrix} -1/\tau - k_1 & 0 & 0 \\ k_1 & -1/\tau - k_2 & 0 \\ 0 & k_2 & -1/\tau \end{bmatrix}, b = \begin{bmatrix} C_{\mathrm{Af}}/\tau \\ 0 \\ 0 \end{bmatrix}$$

Set z = c − c∗, where Mc∗ + b = 0. Then dz/dt = Mz. For fixed inputs within the interval, z(t+Δt) = exp(MΔt)z(t).

$$F_{\Delta t}(c, u; q) = c^* + e^{M\Delta t}(c - c^*)$$

This formula remains valid when T changes at a boundary: compute a new M and c∗ for the next interval, retaining the boundary state.

A worked first transition

At 330 K, $\tau = 5$ min and feed = 1.2 mol/L, $a = 1/\tau + k_1 = 0.4$ and $b = 1/\tau + k_2 = 0.3$ min^{-1}. The steady composition is $c^* = [0.6, 0.4, 0.2]$ mol/L. With $\Delta t = 0.2$ min and $c_0 = [1.2, 0, 0]$:

$$C_{A_1} = C_A^* + \left(C_{A_0} - C_A^*\right) e^{-a\Delta t}$$

$$C_{B_1} = C_B^* + \left(C_{B_0} - C_B^*\right) e^{-b\Delta t} + k_1 \left(C_{A_0} - C_A^*\right) \frac{e^{-a\Delta t} - e^{-b\Delta t}}{b - a}$$

$C_{C,1} = S_1 - C_{A,1} - C_{B,1}$. This gives [1.153870, 0.045672, 0.000458] mol/L.

Forward Euler gives [1.152, 0.048, 0]. It differs because the rates change during the interval. When a = b, the fraction's limit is $\Delta t \exp(-a\Delta t)$.

Concentration changes over the first 0.2 min

At [1.2,0,0], A inflow minus outflow is zero and its reaction consumption is 0.24 mol/(L·min). B forms at that rate; C initially has zero production rate because no B is yet present.

Quantity	Calculation	mol/L
Euler A	$1.2-0.2\times0.24$	1.152000
Exact A	$0.6+0.6\exp(-0.08)$	1.153870
B equilibrium-deviation term	$0.4-0.4\exp(-0.06)$	0.023294
Transfer from A into B	$0.12[\exp(-0.08)-\exp(-0.06)]/(-0.1)$	0.022378
Exact B	0.023294+0.022378	0.045672
Exact C	1.2−1.153870−0.045672	0.000458

Euler uses the initial rate throughout the interval. During the actual interval, A decreases, B forms, and C production begins. These changing rates explain its difference from the exact interval solution.

Transition-pair construction from trajectories

One trajectory has states of shape (151, 3) and controls of shape (150, 3). Split by trajectory, then stack its rows:

```
inputs = np.column_stack((states[:-1], controls))
increments = states[1:] - states[:-1]
# inputs: (150, 6); increments: (150, 3)
x_mean = train_inputs.mean(axis=0)
x_scale = np.maximum(train_inputs.std(axis=0), 1e-8)
d_mean = train_increments.mean(axis=0)
d_scale = np.maximum(train_increments.std(axis=0), 1e-8)
```

The first input row contains c_0 and the first interval control. The last contains c_{149} and the last control; its label is c_{150}. Test statistics do not enter scaling.

Restore the increment and evaluate both loops

$$\Delta c_k = c_{k+1} - c_k$$
$$\hat{c}_{k+1} = c_k + \mu_\Delta + s_\Delta \odot g_\theta\left(\tilde{v}_k\right)$$

```
# Reference current states at every step
one_step = dynamic_step(model, scaling,
                        states[:-1], controls)
# Only the initial state is supplied
recursive = rollout(model, scaling, controls, states[0])
err_one = one_step - states[1:]
err_roll = recursive[1:] - states[1:]
```

Restore Δc before adding it to c. Flatten all noninitial times across test trajectories for overall component RMSE; keep the horizon axis for RMSE_j,h. Both loops use the same future inputs.

Restoring increments and checking total concentration

Take state [0.6,0.4,0.2] mol/L, increment mean [0,0,0], scales [0.02,0.01,0.01] mol/L, and model output [−0.5,0.6,0.4].

Step	Value
Standardized output	[−0.5,0.6,0.4]
Multiply by scales	[−0.010,0.006,0.004] mol/L
Add to current state	[0.590,0.406,0.204] mol/L
Total concentration	0.590+0.406+0.204=1.200 mol/L

```
increment = d_mean + d_scale * standardized_increment
next_state = state + increment
```

These example increments sum to zero. A general MLP does not enforce that property, so check the sum and negative entries at each predicted state. When the initial total differs from feed, compare against the exact total-concentration transition rather than a fixed 1.2.

Derive the error recurrence

$$e_{k+1} = f_\theta\left(\hat{c}_k, u_k\right) - f_\theta\left(c_k, u_k\right) + \delta_k$$
$$\delta_k = f_\theta\left(c_k, u_k\right) - F_{\Delta t}\left(c_k, u_k\right)$$

If the learned map is L_k-Lipschitz between c_k and ĉ_k, then:

$$\left\|e_{k+1}\right\| \le L_k \left\|e_k\right\| + \left\|\delta_k\right\|$$

For constant L and local error bound ε, repeated substitution gives:

$$\left\|e_h\right\| \le L^h \left\|e_0\right\| + \varepsilon \sum_{r=0}^{h-1} L^r$$

L < 1 limits this bound to ε/(1−L) when $e_0 = 0$; L = 1 gives hε. The sensitivity and error bounds must hold along the relevant states. Test RMSE alone is not that bound.

Intermediate states in the two evaluation protocols

Use reference $F(c)=0.9c$, model $f(c)=0.9c+0.001$, and $c_0=1$. Values below are calculated from these scalar functions.

Step	Reference c	One-step prediction	Rollout prediction	Rollout error
0	1	Initial state	1	0
1	0.9	0.901	0.901	0.001
2	0.81	0.811	0.8119	0.0019
3	0.729	0.730	0.73171	0.00271

```
state = 1.0
for step in range(3):
    state = 0.9 * state + 0.001
    print(step + 1, state)
```

The second rollout computes $0.9\times0.901+0.001$. One-step receives reference 0.9 again and computes $0.9\times0.9+0.001$, so it does not inherit the previous error.

Optional: multistep state-transition training

A multi-step loss begins at a reference state, then feeds predicted states forward for H steps and penalizes their trajectory error. Use component scaling consistently.

$$L_H = \frac{1}{NH} \sum_n \sum_h \left\| \hat{c}_{n,h} - c_{n,h} \right\|_2$$

Gradients pass through every unrolled transition. This can expose the model to its own states during training, but increases memory and may make training harder. Construct windows inside train trajectories only; choose H with validation trajectories.

The supplied results use one-step training.

Exercises with short answer checks

1. Why 151 states? A 30 min horizon has 150 intervals of 0.2 min, plus its initial state.
2. How many parameters? 224 + 1056 + 99 = 1379.
3. What if $S_0 \neq$ feed? The reference sum evolves as feed + $(S_0 - \text{feed})\exp(-t/\tau)$; a constant-sum check would be wrong.
4. Error at h = 10 in the scalar example? $0.001(1-0.9^{10})/0.1 \approx 0.006513$.
5. Can we run the model at Δt = 1 min by changing a variable? Its learned output still represents 0.2 min. Apply five steps or train a map for the new interval.
6. What if a reference state is injected halfway through rollout? That is a forecast reset; report the reset times and evaluate the resulting protocol separately.

Optional sequence models and original sources

With incomplete observations, an RNN/LSTM hidden state can summarize available history. An encoder-decoder can convert observed history into a future sequence conditioned on the prescribed controls. Keep future measurements out of the encoder.

- Sutskever, Vinyals & Le (2014), Sections 1-2: sequence encoding and decoding.
- PyTorch RNN and LSTM: recurrent updates, inputs, hidden states, and outputs.
- SciPy matrix exponential: an alternative implementation of $\exp(M\Delta t)$.